

Exception handling

COMPLETE LEARNING GUIDE

What You'll Learn

Robust Java programs anticipate failure — a missing file, a null reference, a division by zero — and exception handling is the language's structured mechanism for detecting, propagating, and recovering from these failures without crashing the whole program or leaving resources in a corrupted state. This guide covers Java's exception class hierarchy and the checked-versus-unchecked distinction that the compiler actively enforces, the mechanics of try-catch-finally including multi-catch blocks, the modern try-with-resources statement that guarantees resources are closed even when an exception is thrown, how to define and throw custom exceptions, and exception chaining for preserving root-cause information. Interviewers frequently probe this topic because sloppy exception handling — swallowing exceptions silently, catching `Exception` too broadly, or leaking resources — is exactly the kind of subtle bug that separates junior code from production-ready code.

1. The Exception Class Hierarchy

Every throwable in Java descends from `Throwable`, which splits into `Error` (serious JVM-level problems like `OutOfMemoryError` that applications generally shouldn't try to catch or recover from) and `Exception` (problems an application is expected to handle). `Exception` further splits into checked exceptions (subclasses of `Exception` but not `RuntimeException`, such as `IOException` and `SQLException`) and unchecked exceptions (subclasses of `RuntimeException`, such as `NullPointerException` and `ArrayIndexOutOfBoundsException`). This hierarchy is what the compiler inspects to decide whether it must force you to handle a given exception.

- `Throwable` !' `Error` (JVM-level, generally not caught) and `Exception` (application-level).
- Checked exceptions (`IOException`, `SQLException`) — must be caught or declared with throws, enforced at compile time.
- Unchecked exceptions (`RuntimeException` and subclasses: `NullPointerException`, `IllegalArgumentException`, `ArrayIndexOutOfBoundsException`) — not compiler-enforced, usually signal programming bugs.
- Checked exceptions represent recoverable external conditions; unchecked exceptions usually represent a bug that should be fixed, not caught defensively.

2. try-catch-finally Mechanics

Code that might throw is wrapped in a try block; one or more catch blocks handle specific exception types, matched top to bottom (so a more specific exception type must be caught before a more general one, or the specific catch becomes unreachable and the compiler rejects it); a finally block runs unconditionally — whether the try succeeded, an exception was caught, or an exception propagated uncaught — making it the traditional place to release resources. A multi-catch block (catch (IOException | SQLException e)) lets one handler cover multiple unrelated exception types without duplicating the handling code, as long as the types don't share a subclass relationship.

JAVA

```
try {
    riskyOperation();
} catch (FileNotFoundException e) {
    System.err.println("File missing: " + e.getMessage());
} catch (IOException | SQLException e) {
    // multi-catch: handles two unrelated exception types identically
    System.err.println("I/O or DB failure: " + e.getMessage());
} finally {
    System.out.println("Runs no matter what happened above");
}
```

3. try-with-resources

Before Java 7, releasing a resource (a file stream, a database connection) safely required a finally block that itself could throw while closing, masking the original exception — a genuinely awkward and error-prone pattern. try-with-resources, introduced in Java 7, automatically closes any resource that implements AutoCloseable at the end of the try block, in reverse order of declaration, even if an exception is thrown — and it does so without any explicit finally block at all. Since Java 9, a resource declared as an effectively-final variable outside the try can be referenced directly inside the parentheses without re-declaring it.

JAVA

```
try (BufferedReader reader = new BufferedReader(new FileReader("data.txt"))) {
    String line;
    while ((line = reader.readLine()) != null) {
        process(line);
    }
} catch (IOException e) {
    System.err.println("Failed to read file: " + e.getMessage());
}
// reader.close() is called automatically here, even if an exception was thrown
```

4. throw vs. throws, and Custom Exceptions

`throw` is a statement that actually raises an exception instance at a specific point in code (`throw new IllegalArgumentException("bad input");`); `throws` is a method-signature declaration listing checked exceptions that method might propagate to its caller, forcing every caller to handle or re-declare them. Custom exceptions are ordinary classes that extend `Exception` (checked) or `RuntimeException` (unchecked), typically adding a constructor that forwards a message (and optionally a cause) to the superclass — this lets a domain express specific, meaningful failure types (`InsufficientFundsException`) instead of forcing every caller to parse a generic exception's message string to figure out what went wrong.

JAVA

```
public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public void withdraw(double amount) {
    if (amount > balance) {
        throw new InsufficientFundsException("Requested " + amount + " exceeds
balance " + balance);
    }
    balance -= amount;
}
```

5. Exception Chaining and Preserving Root Cause

When catching one exception and throwing a different, higher-level one (common when translating a low-level `SQLException` into a domain-specific `DataAccessException`), it's essential to pass the original exception as the cause parameter rather than discarding it — otherwise the stack trace shown in logs points only at the translation code, hiding the actual root cause that triggered the failure. `getCause()` lets any later code (or a log viewer) walk back through the full chain of causation, which is often the only way to diagnose a failure that passed through several layers of a real application.

JAVA

```
try {
    executeQuery();
} catch (SQLException e) {
    throw new DataAccessException("Failed to load user", e); // e becomes the
    "cause"
```

6. finally, Return Values, and a Subtle Gotcha

A `return` statement inside a `try` or `catch` block does not exit the method immediately if a `finally` block is present — `finally` still runs first. More subtly, if `finally` itself contains a `return` statement, it silently overrides any return value or even any exception from the `try/catch` block, discarding them entirely with no warning — this is almost always unintentional and is flagged by every reputable linter, but it's a genuinely correct (if surprising) part of the language specification worth knowing for interview questions that test edge-case

understanding.

7. Best Practices: What Not To Do

The single worst exception-handling anti-pattern is the empty catch block — `catch (Exception e) {}` — which silently discards a failure with no log, no rethrow, and no trace of what happened, turning debugging into guesswork. A close second is catching the overly broad `Exception` (or worse, `Throwable`) type when a specific exception type was expected, which can accidentally swallow unrelated bugs (like a `NullPointerException` from a typo) alongside the exception you actually meant to handle. Production code should log the full exception (including stack trace) at the point of catching, or rethrow it wrapped with additional context, but never discard it silently.

Common Pitfalls

- Writing an empty catch block that silently swallows an exception, making failures invisible and debugging nearly impossible.
- Catching the broad `Exception` type when a specific one was intended, accidentally masking unrelated bugs alongside the expected failure.
- Forgetting that a `return` inside `finally` silently discards any exception or return value from the `try/catch` block.
- Declaring `throws Exception` on a method signature as a lazy way to avoid the compiler's checked-exception enforcement, losing all type information for callers.
- Catching an exception and rethrowing a new one without passing the original as the cause, losing the root-cause stack trace in logs.
- Manually closing resources in a `finally` block instead of using `try-with-resources`, risking a resource leak if the `close()` call itself throws.
- Using exceptions for ordinary control flow (e.g. throwing to break out of a loop) instead of a normal conditional, hurting both performance and readability.

Key Takeaways

- Checked exceptions are compiler-enforced and represent recoverable conditions; unchecked (`RuntimeException`) exceptions usually signal a programming bug.
- `finally` always runs, whether or not an exception was thrown — the correct place for cleanup that isn't resource closing.
- `try-with-resources` auto-closes any `AutoCloseable` resource, even on an exception, and should be preferred over manual `finally`-based closing.
- `throw` raises an exception instance; `throws` declares that a method may propagate one — don't confuse the two keywords.
- Always chain exceptions (pass the original as the cause) when translating one exception type into another, to preserve root-cause traceability.

- An empty catch block is the single worst exception-handling anti-pattern — always log or rethrow, never silently discard.

Learn More

Oracle Java Tutorials — Exceptions

<https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>

Oracle Java Tutorials — The try-with-resources Statement

<https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html>

Baeldung — Exception Handling in Java

<https://www.baeldung.com/java-exceptions>